Phase 4: Build & Benchmark a High-Frequency Trading System in C++

100 Points Possible

4/28/2026

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

Details

Phase 4: Build & Benchmark a High-Frequency Trading System in C++

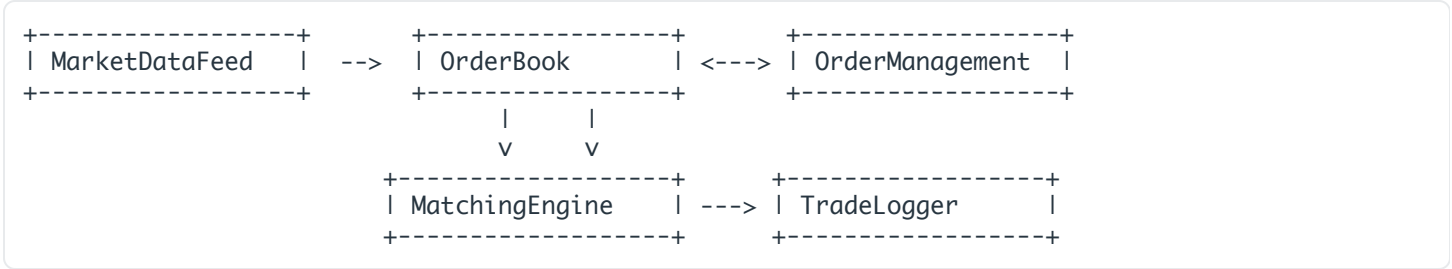
🎯 Project Objective

Design and implement a **performance-optimized HFT trading system prototype** in C++. The system will:

- Ingest market data
- Accept and manage client orders
- Match orders in a fast limit order book
- Report trades
- Measure and analyze **tick-to-trade latency**

Apply advanced C++ concepts such as smart pointers, memory pools, templates, compile-time checks, and profiling tools.

🧱 System Architecture Overview



Each module will be instrumented to benchmark tick-to-trade latency.

🧩 Module-by-Module Guide

1️⃣ Market Data Feed Simulator

Tasks:

- Simulate ticks with mock price data (`symbol`, `bid`, `ask`)
- Use `alignas(64)` to enforce cache-line alignment
- Use `std::chrono::high_resolution_clock` for timestamps

```
struct alignas(64) MarketData {
    std::string symbol;
    double bid_price;
    double ask_price;
    std::chrono::high_resolution_clock::time_point timestamp;
};
```

2️⃣ OrderBook (Template-Based)

Tasks:

- Create a templated `Order<TPrice, TOrderID>`
- Store orders using `std::unique_ptr`
- Use a memory pool allocator for performance
- Store buy/sell orders using `std::multimap` or a cache-friendly structure

```
template <typename PriceType, typename OrderIdType>
struct Order {
    OrderIdType id;
    PriceType price;
    int quantity;
    bool is_buy;
};
```

3️⃣ Order Management System (OMS)

Tasks:

- Track new, cancel, partial, filled states

- Use `shared_ptr` if orders are accessed in multiple places
- Replace raw new/delete with RAII and smart pointers
- Use `static_assert` to restrict allowed types (e.g., order ID must be integral)

```
static_assert(std::is_integral<OrderIdType>::value, "Order ID must be an integer");
```

4️⃣ Matching Engine

Tasks:

- Match buy/sell orders in the book
- Return matched trades
- Optimize for cache locality
- Benchmark tick-to-trade latency on every match

5️⃣ Trade Logger

Tasks:

- Store trades in `std::vector` using `reserve()` for preallocation
- Write logs in batches
- Use RAII for safe resource management

📏 Tick-to-Trade Latency Testing

⏱️ What is Tick-to-Trade?

Time between:

- Receiving a price update (**tick**) and
- Sending a trade order (**trade**)

🔧 How to Measure It

```
using Clock = std::chrono::high_resolution_clock;
auto start = Clock::now();

// Simulate tick reception
marketDataHandler.handleTick(tick);

// Match & trade generation
matchingEngine.matchOrder(order);

auto end = Clock::now();
auto latency = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start).count();
```

Store results in a `std::vector<long long>` for statistical analysis.

📊 Latency Analysis

```
void analyzeLatencies(const std::vector<long long>& latencies) {
    if (latencies.empty()) return;

    auto min = *std::min_element(latencies.begin(), latencies.end());
    auto max = *std::max_element(latencies.begin(), latencies.end());
    double mean = std::accumulate(latencies.begin(), latencies.end(), 0.0) / latencies.size();
    double variance = 0.0;
    for (auto l : latencies) variance += (l - mean) * (l - mean);
    double stddev = std::sqrt(variance / latencies.size());
    long long p99 = latencies[static_cast<int>(latencies.size() * 0.99)];

    std::cout << "Tick-to-Trade Latency (nanoseconds):\n";
    std::cout << "Min: " << min << "\nMax: " << max << "\nMean: " << mean
              << "\nStdDev: " << stddev << "\nP99: " << p99 << "\n";
}
```

🧪 Experiments to Run

Variable	Experiment	What to Observe
Smart vs raw pointers	Swap `unique_ptr` with raw pointers	Memory safety vs overhead
Memory alignment	Add/remove `alignas(64)`	Cache behavior differences
Custom allocator	Use memory pool vs `new/delete`	Allocation speed
Container layout	Flat array vs `map/multimap`	Access time difference
Load scaling	1K, 10K, 100K ticks	Latency consistency under pressure

---

📝 Final Deliverables Checklist

Deliverable	Required
🗃️ Modular source code (`.hpp/.cpp`)	✅
📝 README (design overview, build/run instructions)	✅

Deliverable	Required
📊 Benchmark report (tick-to-trade latency statistics)	✅
🧪 Performance test results + brief analysis	✅
📈 Architecture diagram (system/module/class flow)	✅
🎥 Video demo showing the system working	✅

🗂️ Suggested File Structure

```
hft_project/
├── include/
|   ├── MarketData.hpp
|   ├── Order.hpp
|   ├── OrderBook.hpp
|   ├── MatchingEngine.hpp
|   ├── OrderManager.hpp
|   ├── TradeLogger.hpp
|   └── Timer.hpp
├── src/
|   ├── MarketData.cpp
|   ├── OrderBook.cpp
|   ├── MatchingEngine.cpp
|   ├── OrderManager.cpp
|   ├── TradeLogger.cpp
|   └── main.cpp
├── test/
|   └── test_latency.cpp
├── CMakeLists.txt
└── README.md
```

📄 Sample Code Snippets

`include/Order.hpp`

```
#pragma once
#include <string>
#include <memory>

template <typename PriceType, typename OrderIdType>
struct Order {
    OrderIdType id;
    std::string symbol;
    PriceType price;
    int quantity;
    bool is_buy;

    Order(OrderIdType id, std::string sym, PriceType pr, int qty, bool buy)
```

```
        : id(id), symbol(std::move(sym)), price(pr), quantity(qty), is_buy(buy) {}
};
```

include/Timer.hpp

```
#pragma once
#include <chrono>

class Timer {
public:
    void start() {
        m_start = std::chrono::high_resolution_clock::now();
    }

    long long stop() {
        auto end = std::chrono::high_resolution_clock::now();
        return std::chrono::duration_cast<std::chrono::nanoseconds>(end - m_start).count();
    }

private:
    std::chrono::high_resolution_clock::time_point m_start;
};
```

src/main.cpp

```
#include <iostream>
#include <vector>
#include <numeric>
#include <algorithm>
#include "../include/Order.hpp"
#include "../include/Timer.hpp"

using OrderType = Order<double, int>;

int main() {
    std::vector<long long> latencies;
    const int num_ticks = 10000;

    for (int i = 0; i < num_ticks; ++i) {
        Timer timer;
        timer.start();

        // Simulated tick + order match (replace with real logic)
        OrderType order(i, "AAPL", 150.0 + (i % 5), 100, i % 2 == 0);
        // simulate match logic here

        latencies.push_back(timer.stop());
    }

    // Analyze latency
    auto min = *std::min_element(latencies.begin(), latencies.end());
    auto max = *std::max_element(latencies.begin(), latencies.end());
    double mean = std::accumulate(latencies.begin(), latencies.end(), 0.0) / latencies.size();

    std::cout << "Tick-to-Trade Latency (nanoseconds):\n";
    std::cout << "Min: " << min << " | Max: " << max << " | Mean: " << mean << '\n';
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)
project(HFT_Project)

set(CMAKE_CXX_STANDARD 17)

include_directories(include)

add_executable(hft_app
    src/main.cpp
    src/MarketData.cpp
    src/OrderBook.cpp
    src/MatchingEngine.cpp
    src/OrderManager.cpp
    src/TradeLogger.cpp
)
```

Keep in mind, this submission will count for everyone in your Q22026 Group group.

Choose a submission type

Text | Upload | More

Submit Assignment